12 MANAGING DECISION UNCERTAINTY USING RANDOM FORESTS

12.1 Decision-Making Under Uncertainty

12.2 Features of Decision Uncertainty

12.3 Backorder and Its Implications

12.4 Machine-Learning Options to Aid in Decision-Making Under Uncertainty

12.4.1 Components of a Decision Tree Model

12.4.2 Classification Trees

12.4.3 Advantages, Disadvantages, and Ways to Improve Classification Trees

12.5 Random Forest

12.6 Backorder Prediction Using Random Forests

12.6.1 Data Import and Selection

12.6.2 Imbalance in the Data

12.6.3 Hyperparameter Tuning

12.6.4 Testing the Performance of Our Random Forest Model

12.6.4.1 Accuracy and ROC Values

12.6.4.2 Confusion Matrix, Sensitivity, and Specificity

12.6.4.3 Role of Downsampling

12.7 Business Insights and Summary

12.8 Implementation Using R: Backorder Prediction

12.9 Understanding the Chapter

LEARNING OBJECTIVES

1. Understand the importance of efficient business decisions, anticipating demand
2. Learn decision-making under uncertainty

3. Apply decision tree models to predict uncertain decisions
4. Review the advantages and limitations of tree models
5. Employ boosted decision trees to predict stocking decisions

12.1 DECISION-MAKING UNDER UNCERTAINTY

Decision-making under uncertainty has been studied as a continuum of decisions that are made when information is missing or is available only in probabilistic form (e.g., the likelihood of success is 80%).[1] Consider the following two scenarios that highlight the inherent uncertainty in business decision-making.

In the first scenario, a major airline, considered a dominant player at one of its hub airports, is faced with a new challenge: the entry of a low-cost, no-frills competitor. It has to decide whether to react to the new entrant (action) or not do anything (inaction). Both decisions carry inherent uncertainties. If it decides to take an action and reduce price, there are several uncertain outcomes. First, will a price reduction deter customers from flocking to the new entrant? Second, will it actually increase market share, or will it erode profits? Third, will the price drop strengthen its brand image, or will it be seen as a sign of weakness? Fourth, actions like price reductions can sometimes lead to price wars, which might hurt the industry as a whole. Alternatively, the airline can maintain the status quo, but this inaction has its own uncertainties. First, how many existing customers might be persuaded away by the competitor's lower prices or novel offerings. Second, staying the course might be perceived by stockholders as either a confident move, showcasing the brand's strength, or indicating an unwillingness to adapt to changing market dynamics. Third, by not adjusting to the changing market dynamics, there's a potential loss of opportunities that the new entrant might exploit. Fourth, will the business be able to recover its position if it realizes too late that a response was necessary?

In the second scenario, an electronics retailer is preparing to stock a new smartphone. Due to supply chain issues from the manufacturer, the retailer learns that they will receive only half of the originally ordered units triggering the following decision dilemma: Should the retailer accept backorders, allowing customers to reserve now and receive the product later when more stock arrives, or should they sell only the available stock and potentially lose sales to competitors? Both decisions have their inherent uncertainties.

First, it is unclear how many customers will prefer immediate purchase compared to those willing to wait for a backordered item. Second, offering backorders might be seen as a service, but extended wait times can negatively affect customer satisfaction, leading to refunds and poor customer reviews. Third, if the timeline for additional shipments from the manufacturer is not clear, it adds to the uncertainty. A short delay may be acceptable to customers, but longer ones can affect sales and customer trust. Fourth, information (or lack of information) about the stock availability at competitor outlets can add to the uncertainty. If competitors also do not have sufficient stock, not offering backorders might result in lost sales.

These scenarios highlight business decisions with inherent uncertainty. Other decisions fraught with uncertainty include, for instance, (1) deciding between offering price discounts to encourage growth or increasing prices to increase profitability, (2) decisions about allocating resources to retain current customers versus recruit new ones, (3) whether to invest in an immediate profitable venture or focus on long-term projects that might yield larger returns later, or (4) projecting the impact of weather and other natural events on the supply chain that can disrupt transportation, delay deliveries, and affect raw material availability.

12.2 FEATURES OF DECISION UNCERTAINTY

Uncertain decisions have several important features that need to be considered.[2] First, what is the value in taking a decision under uncertainty versus gathering more information before proceeding? Let's take the example of a pharmaceutical company that is performing drug trials for a new cancer treatment. It will use randomized controlled trials (RCTs) to control for all factors and find the influence of the focal drug on cancer treatment; this is generally referred to as internal validity. However, the uncertainty that the company has to face is how the drug will fare when patients with different comorbidities, of different races, or with any other characteristics that makes them respond to the drug differently, actually take the drug after it has been introduced in the market; this is generally referred to as external validity. Therefore, the company has to weigh the value of introducing a beneficial drug against the risk associated with such an introduction.

A second feature of uncertain decisions is to determine when and how to respond. Consider the earlier example of a major airline when a low-cost competitor enters the scene. The airline must decide when and by how much to modify its fares and whether to introduce new offers. The decision resembles a balancing act between ensuring profitability and incurring costs from strategy changes. To determine the best course of action, companies often conduct sensitivity analyses. This involves tweaking different decision variables and observing the potential impact on outcomes at different points in time. By doing so, the airline can gauge which factors, when and to what degree, might necessitate a change in their initial decision.

A third feature of uncertain decisions is to determine which options to follow: Should the business stick to conventional options or explore new ones? For instance, in our earlier example when an electronics retailer is confronted with a backorder situation for a popular new smartphone, the conventional approaches are to either sell the available stock or continue accepting backorders. However, given the uncertainties and potential customer dissatisfaction, they might consider a new strategy: offering discounts to those customers willing to backorder and wait for delivery. While this strategy might entail a short-term reduction in profit, it has the potential to increase long-term customer loyalty and mitigate dissatisfaction from extended wait times.

To reduce decision uncertainty, prediction models are used that simulate different potential situations and how these situations will impact different aspects of the business. In this chapter, we will examine how machine-learning algorithms can be used to solve problems in supply chain management, especially when decisions need to be made with uncertain information. One such decision-making under uncertainty is identifying products that will be backordered.

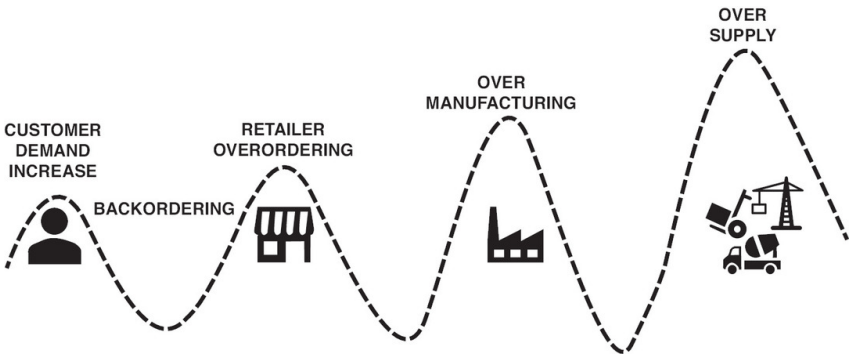
12.3 BACKORDER AND ITS IMPLICATIONS

Beginning in 2020, the microchip industry faced supply chain issues because of the pandemic, labor shortages, and geopolitical unrest. This resulted in many technological products going out of stock. Consumers who placed orders were left waiting, sometimes indefinitely, because the firms were not sure when they would receive the chips to fulfill these orders. The Defense Logistics Agency of the U.S. Department of Defense, which manages the global defense supply chain, faces similar challenges. It routinely takes steps to maximize combat readiness by minimizing backorders.[3]

Backorder can be defined as an order of a product that a firm cannot fill due to the product's unavailability but expects to fill at a future date. Backorders result in disappointed consumers and reduced profitability for firms as backorders can deter consumers from purchasing the product. Recent research has demonstrated that consumers whose products were backordered are far less likely to purchase the product in the future compared to consumers whose products were not backordered.[4] The longer the backorder lasts, the greater the drop in future purchases. Hence, businesses try their

level best to avoid the situation of backorders piling up because of its long-term impact on customer satisfaction, ability to retain customers, and firm profitability. One commonly recommended way to avoid backorders is to accurately forecast demand. However, that is easier said than done because several factors outside the influence of the business introduce uncertainties in forecasting, such as changing customer tastes that make demand uncertain, economic upturns and downturns, gross domestic product (GDP), geopolitical conflicts, unrest, or a pandemic. Firms do not want to lose customers or lose their trust by not being able to fulfill an order on time. However, the enormous pressure that firms face to ensure timely delivery of products, especially when the firm itself is reliant on other suppliers for parts, is not trivial. This pressure can result in increased labor and production costs, overstocking to prevent shortages, and high shipment expenses.

Backorders also cause the *bullwhip* effect, which occurs when a small change in demand is overestimated and communicated through the supply chain to retailers, wholesalers, distributors, and eventually manufacturers. However, if the demand is temporary, it dies out, causing an oversupply of some products. We witnessed this during the pandemic when people staying home started remodeling and a huge demand for furniture ensued, resulting in furniture being severely backordered. To make up for the increased demand, firms manufactured more furniture. However, with the easing of the pandemic restrictions, the demand for furniture went down, resulting in an oversupply of furniture. This is a classic example of the bullwhip effect depicted in Figure 12.1.



Description

Figure 12.1 Bullwhip Effect

However, we need to keep in mind one important factor when considering backorders. Although estimating which products are likely to be backordered may be one of the most uncertain situations to predict and can cause a high level of disruption for the business and its customers, it is not an issue that businesses face routinely or at predictable intervals. It is possible that a business may not have faced a backorder issue for several years. Hence, the data the business needs to predict backorders may be imbalanced. That is, businesses may not have a very good method to predict why and when they might face a backorder issue. We know that the predictive model is only as good as the data it is trained on. Hence, having imbalanced data can affect the quality of predictions. Machine-learning methods are now routinely used to tackle the problem of backorder forecasting, given their ability to analyze large structured and unstructured datasets that are unbalanced.

We next present a tree-based machine-learning method that has been used to estimate backorders so that strategies can be devised to address this supply chain issue.

12.4 MACHINE-LEARNING OPTIONS TO AID IN DECISION-MAKING UNDER UNCERTAINTY

Machine learning, especially tree-based models, offer businesses a structured way to make decisions under uncertainty. These models segment data based on different conditions, providing insight into complex scenarios. For instance, the airline faced with competition from a low-cost carrier could use decision trees to analyze historical data on passenger preferences, price sensitivities, and other factors to predict potential outcomes. Similarly, the electronics retailer uncertain about how to manage stock for a new smartphone could deploy tree-based models to evaluate previous product launches and understand customer behavior patterns in the face of backorders. Through these models, businesses can simulate various decision pathways, assess probable outcomes, and make more informed choices even in the presence of uncertainty.

12.4.1 Components of a Decision Tree Model

Decision trees are *supervised algorithms* that require labeled data (i.e., data where the relationship between the predictor and the outcome variable is available). Decision trees can be used for both regression tasks and classification depending on whether the outcome variable is continuous or categorical. The term *classification and regression tree* (CART) captures this ability of tree models (we will use the terms *decision tree* and *tree model* interchangeably, but they mean the same thing). For backorder predictions, we will focus on classification trees.

Before we discuss how classification trees make predictions, let's understand the different components of a decision tree as depicted in Figure 12.2.

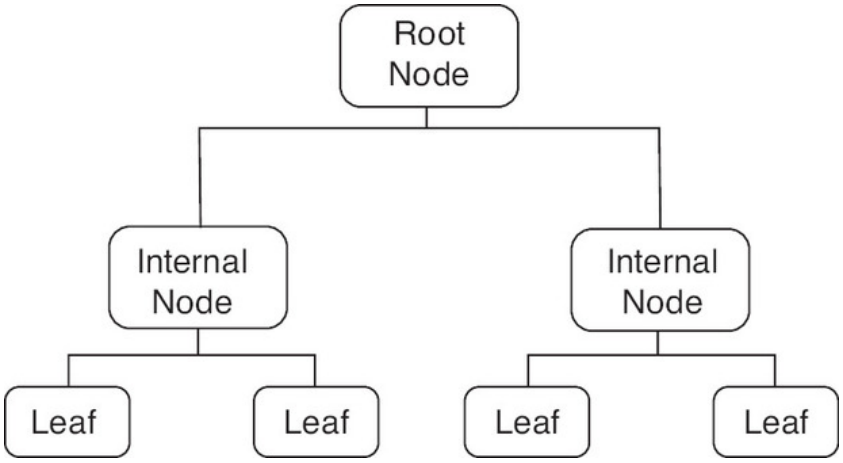


Figure 12.2 Components of a Decision Tree

The *root node* is the first node in a decision tree. This also represents the predictor variable or feature that is split first to divide the entire data into two sets.

The *terminal node* or *leaf* is the final node of a decision tree that cannot be divided further. These terminal nodes contain the mean value of the outcome variable (if the outcome is a continuous variable) or the majority class (if the outcome is a classification task).

The *internal/decision node* is a subnode of the tree between the root node and the terminal node. At an internal or decision node, the data are further divided into new internal nodes or into leaf nodes.

A *branch* represents the connection or relationship between a root node and an internal node, between internal nodes, or between an internal node and a leaf node.

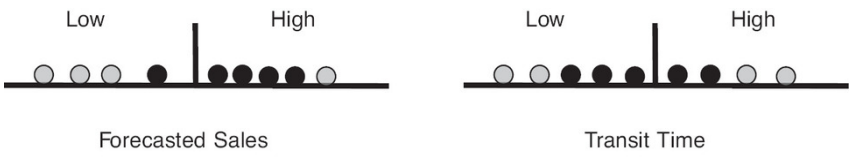
The *depth* of a tree is the longest distance between the root node and any terminal node (leaf). The tree in Figure 12.2 has a depth of 2.

12.4.2 Classification Trees

Consider a simple illustration of backorder prediction. Assume our dataset has two predictor variables: *forecasted sales* and *transit time for the product to arrive*. Both are categorical variables with two levels: low and high. That is, the forecasted sales for a product can be low or high, and the transit time for the product to arrive from the manufacturer to a retailer can also be low or high. The outcome variable is whether the product was backordered (*yes* or *no*). Our goal is to come up with a simple prediction model that tells us if a product will be backordered based on its forecasted sales and transit time. The data are presented in Table 12.1.

Forecasted sales	***Transit time***	***Backordered***
Low	Low	No
High	Low	Yes
Low	Low	No
High	High	Yes
Low	Low	Yes
High	High	No
High	Low	Yes
Low	High	No
High	High	Yes

The first challenge in building a classification tree is to choose a variable that first splits the root node. Let's consider Figure 12.3. The dark circles represent when the product was backordered, and the light circles represent when it was not backordered. We see that the variable of forecasted sales performs better at classifying whether a product was backordered. In the low-forecast group, three out of the four data points are not backordered, and in the high-forecast group, four out of the five data points are backordered. In an intuitive way, high- and low-forecast groups are able to segregate backordered and not-backordered data points more effectively than high- and low-transit-time groups. But this is just a visual assessment. How can we quantify which variable is a better classifier? Gini impurity provides us with one way to do this. Each predictor variable at each level is 1 – (fraction of *backordered* = *"Yes"* datapoints)^2 − (fraction of *backordered* = *"No"* datapoints)^2. Lower values of Gini impurity are preferred over higher values. Let's understand how to calculate the Gini index for the example we are working with.

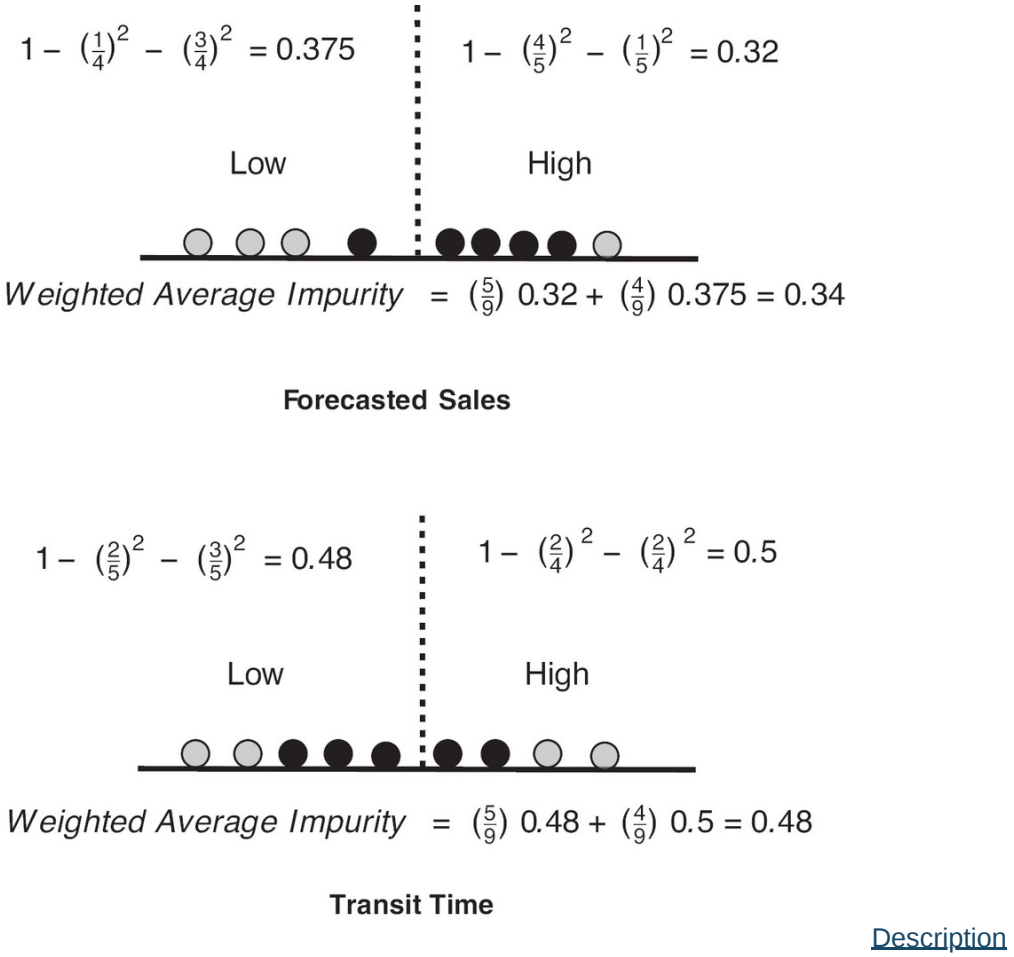


Description

Figure 12.3 Gini Index Calculation

For forecasted sales as shown in Figure 12.3, we have five data points with high-forecast sales and four data points with low-forecast sales. Among the five data points in the high-forecast group, four are for backorders, and one is not. The fraction of items backordered in this group is $\frac{4}{5}$, and the fraction of items not backordered is $\frac{1}{5}$. The Gini impurity for this group is $1 - \left(\frac{4}{5}\right)^2 - \left(\frac{1}{5}\right)^2 = 0.32$. Among the four data points in the low-forecast group, one was backordered, and three were not. The fraction of items backordered in this group is $\frac{1}{4}$, and the fraction of items not backordered is $\frac{3}{4}$. The Gini impurity for this group is $1 - \left(\frac{1}{4}\right)^2 - \left(\frac{3}{4}\right)^2 = 0.375$. Now we can calculate Gini weighted impurity for forecasted sales. Since there are nine data points, the weighted Gini impurity for the low-forecast group will be $\frac{5}{9}$, and for the high-forecast group it will be $\frac{4}{9}$. The weighted average will be $\left(\frac{5}{9}\right)0.32 + \left(\frac{4}{9}\right)0.375 = 0.34$.

Similarly, we can calculate weighted Gini impurity for the transit time variable (as depicted in Figure 12.4).

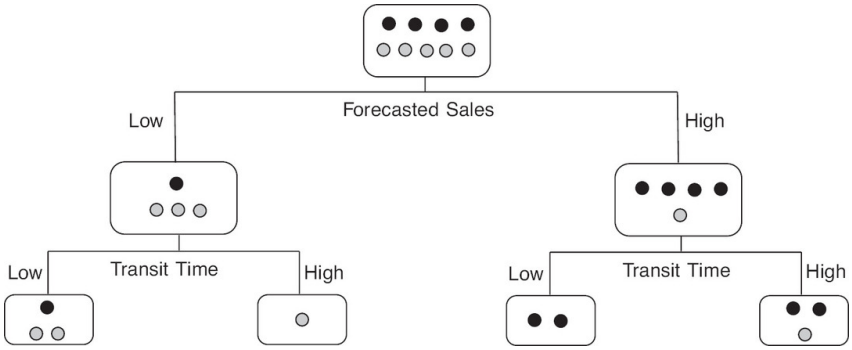


Description

Figure 12.4 Gini Impurity for Forecasted Sales and Transit Time

We find that Gini impurity is lower with forecasted sales; hence, it becomes the variable to split the root node. The resulting decision tree will look like Figure 12.5, in which it splits further based on the transit

time.



Description

Figure 12.5 Decision Tree Splits

Since we have only two predictor variables, this will not be split further, and the final nodes will be the terminal nodes or leaves. Predictions associated with each leaf will reflect the majority class. As an example, if the majority class in the leftmost leaf is *no* (2) and the minority class is *yes* (1), then the prediction associated with this leaf will be *no*.

12.4.3 Advantages, Disadvantages, and Ways to Improve Classification Trees

Classification trees offer many advantages. First, several types of predictor variables (numeric or categorical in nature) can be included in the analysis, and they don't need to be scaled or standardized. Second, outliers don't influence classification tree predictions; therefore, no outlier removal is necessary before analysis. Third, trees are not affected by the presence of irrelevant predictors. If an irrelevant predictor is included in the analysis, the analysis is not affected even when the user is uncertain which predictors have a significant influence. Classification trees can also handle the presence of correlated predictors (multicollinearity), which is very common in real-world data since many of the predictors that users might identify as being important for the analysis could be correlated with one another. Fourth, interactions among predictor variables are automatically incorporated in the analysis; therefore, there is no need to prespecify interactions, which is necessary in methods such as ordinary least squares (OLS) regression and logistic regression.

However, there are some *disadvantages* of using classification trees. Individual trees do not have as much predictive power as many other methods such as logistic regression or neural nets. Trees are also very sensitive to variations in the training data. In other words, if we divide the training data randomly into two parts and train two different trees, then there is a high likelihood that both trees will fail to make similar predictions.

One way to improve the predictive power of trees and reduce their sensitivity to variations in the training data is to combine the predictions of many trees. The intuition behind combining various trees can be understood through the example of measuring temperature. Imagine we have a thermometer that has some random error. Every time we measure temperature, it gives slightly different readings. One way to obtain a more stable and reliable estimate of temperature is to take several measurements of temperature and then average them. In a similar vein, when we combine predictions of various trees, we reduce variation in their predictions and improve their predictive power. The combination of several trees is known as an ensemble of trees; since an ensemble of trees is a forest, the method that creates this ensemble is called random forest.

12.5 RANDOM FOREST

The first step builds multiple distinct trees by using bootstrap samples of training data. *Bootstrapping* is a process of sampling data points from the original dataset with replacement. Imagine we have 30 balls in a box and each of them has a distinct number written on it. In sampling with replacement, we first randomly pick a ball, write down the number written on the ball, and put it back in the box. We again pick a ball, write down the number, and put it back in the box. We are allowed to randomly pick the same ball more than once. Repeating this process 30 times will give us our first bootstrap sample. For a dataset, bootstrap samples are formed by sampling rows with replacement from the original dataset. Since we are replacing each row, there is a possibility that some rows will be picked up more than once, that our bootstrap sample will have duplicate rows of data, and that some rows from the original dataset will be missing in our bootstrap sample. This is exactly what we want because this will help us build diverse trees. This process of building trees from several bootstrap samples and combining their prediction is called *bagging* (short for *bootstrap aggregation*).

In a regular decision tree, all the predictor variables are considered at each split; however, this leads to a problem. If there is a predictor variable that is very good at predicting the outcome variable, then regardless of how many bootstrap samples we use, there is a very high chance that all the trees will look the same and their predictions will be very correlated. In other words, we will fail to build diverse trees. The second step of random forest, therefore, goes beyond simple bagging. It combines bagging with an idea known as *subspace sampling* (also referred to as *feature bagging*). At each split, it randomly selects a subset of predictor variables that are the only ones considered; then a new subset of predictors is considered at the next split from the remaining predictors. So if there are 20 predictor variables, then 5 are randomly chosen at each split. Their performance on a criterion like Gini impurity is compared, and the best-performing variable splits the data. At the next split, a new random sample of 5 predictors from the remaining set of 19 predictors is chosen. This method ensures that if a strong predictor exists, then it is not always chosen to split the data. This way, we get diverse trees from bootstrap samples.

In the third step of random forest, each tree makes its prediction, and the final prediction is based on the consensus value. The outcome that receives the most votes is the final prediction. As an example, if we build 100 trees for a new product where the outcome is unknown, and out of these 100 trees 65 trees predict that the product will be backordered and 35 predict that the product will not be backordered, then the final prediction of the random forest will be *yes*.

As can be noticed, we have not discussed any method to find the optimum number of predictor variables randomly sampled in the second step. There is no way to estimate the optimum number from the dataset. Therefore, this becomes a hyperparameter that needs to be tuned using cross-validation. Another hyperparameter is the number of trees we want to grow. This also needs to be tuned using cross-validation.

12.6 BACKORDER PREDICTION USING RANDOM FORESTS

Let’s assume that a manufacturer of small machines has set up an inventory system to ensure that it can keep track of obtaining machined parts from its suppliers so it can build its machines and deliver them on time to its customers. However, recently, the manufacturer has noticed some backorder issues, leading to dissatisfied customers. To prevent such backorder issues from occurring in the future, the manufacturer uses random forests to build a model for predicting (and hence addressing) backorders. Let’s understand the process and perform the analysis ourselves. We will refer to the business as the manufacturer, and the machines it manufactures as products.

12.6.1 Data Import and Selection

The variables in our analysis include the following.[5] The outcome variable indicated whether a product was backordered (*yes* vs. *no*). The random forest was provided with 16 predictor variables such as the current level of inventory; usual and current transit time; sales forecasts for the next 3, 6, and 9 months; sales in the past 3, 6, and 9 months; minimum recommended stock; and machined parts due from the manufacturer's suppliers.

We begin our analysis by importing the data and checking the class of nonnumeric variables. We want to ensure that nonnumeric or categorical variables are recognized as factors in the analysis.

As we discussed earlier, backorder data tend to have a class imbalance, where the number of items that were backordered is proportionately less than the number of items that were not backordered. Let's check this in our data through a simple plot (Figure 12.6).

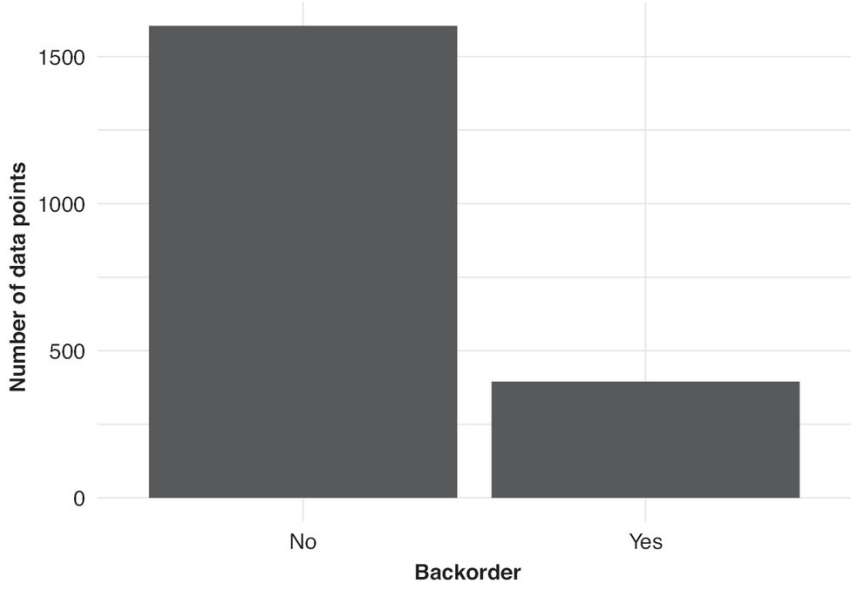


Figure 12.6 Data Imbalance

The plot shows that products that were backordered (i.e., when the outcome variable was *yes*) represented approximately $\frac{1}{5}$ of the data. This makes the times the product was not backordered (i.e., when the outcome variable was *no*) a majority level (also referred to as majority class). *Yes* becomes a minority level. It is necessary to use a sampling method that provides an appropriate representation to the minority level. Next we discuss one such sampling method.

12.6.2 Imbalance in the Data

To handle the issue of class imbalance, we use a downsampling method that samples rows from the training data in such a way that the majority level (when the products were not backordered—i.e., when outcome variable = *no*) is sampled less. This increases the proportion of the minority level (when the products were backordered) in model training. We set the train data in such a way that the backorders are at most 1.5 times as many rows as minority levels. If we compare this situation to our

base situation when no sampling is applied, we will have majority levels with 4 times as many rows as minority levels.

However, having corrected for the imbalance in the training data so that the model has enough data points from which to learn how backorders are affected by the predictors, we need to ensure that the model is tested on a dataset that captures the real world. The real world has a data imbalance, where backorders are fewer in number; therefore, we need to have test data that reflect this smaller number of backorders compared to all orders so that the efficacy of the model in the real world can be tested. Thus, in the test data, no undersampling is used, and the imbalance is retained.

12.6.3 Hyperparameter Tuning

We formulate the model with backorder as the outcome variable and all other variables as predictors. We then specify the random forest algorithm and also the hyperparameters—such as the number of predictors randomly sampled at each split when a tree is created, the number of trees contained in a random forest, and the minimum number of data points in a node required for the node to be split further (please see the Technical Appendix for a detailed description of hyperparameters). Each hyperparameter is then tuned through a grid search. Since our outcome variable is categorical (*yes* and *no*), we need to ensure that our model type is specified as a classification model.

We use fivefold cross-validation, indicating that there will be five partitions of the training dataset and that the performance of each hyperparameter combination will be assessed using this cross-validation procedure. Moreover, we specify 50 different hyperparameter combinations. For each of the 50 hyperparameter combinations, a model is fit on the four partitions, and its performance is assessed on the fifth partition. The AUC ROC (area under the curve receiver operating characteristics) curve and overall accuracy of the model are computed (please refer to Chapter 9 on logistic regression, which provides a detailed description of interpreting AUC ROC values for a model).

We then pick the best hyperparameter combination.

The analysis reveals that the best hyperparameter combinations are the following: the number of predictors randomly sampled at each split when a tree is created = 5, the number of trees contained in a random forest = 69, and the minimum number of data points in a node that is required for the node to be split further = 31.

12.6.4 Testing the Performance of Our Random Forest Model

We fit the model using the best hyperparameter combination on the entire training set and then validate its performance on the test data. The accuracy and ROC values (Table 12.2), along with the confusion matrix for the test data (Table 12.3), are reported.

12.6.4.1 *Accuracy and ROC Values*

.metric	.estimator	.estimate	.config
accuracy	binary	0.8223553	Preprocessor1_Model1
roc_auc	binary	0.9076335	Preprocessor1_Model1

	No	Yes
No	329	16
Yes	73	83

The results show that the accuracy is 82.2%, and the AUC is 0.908. Given the imbalance in levels of the outcome variable and the fact that the minority level (when the products were backordered) is more important for the prediction task, we need to estimate the sensitivity as well as the specificity of this prediction.

12.6.4.2 *Confusion Matrix, Sensitivity, and Specificity*

A quick recap from Chapter 9 on logistic regression explains the confusion matrix. Let's work with the following example to understand the process of calculating sensitivity and specificity.

	Truth	
Prediction	No	Yes
No	i	iii
Yes	ii	iv

Given the confusion matrix in Table 12.4, sensitivity and specificity are as follows:

$$Sensitivity = \frac{iv}{iii + iv} \text{ and } Specificity = \frac{i}{i + ii}$$

Intuitively, sensitivity is the true positive rate and represents the ability of our model to correctly classify products that were backordered. Specificity is the true negative rate and represents the ability of our model to correctly classify products that were not backordered. If sensitivity of our model is low, then we will not be able to identify ahead of time products that are likely to be backordered. On the other hand, if specificity is low, then we will end up overordering products that are less likely to be backordered. For our model, sensitivity is $\frac{83}{83+16} = 0.838$, and specificity is $\frac{329}{329+73} = 0.818$.

12.6.4.3 *Role of Downsampling*

In the preceding analysis, we used *downsampling* to handle the issue of class imbalance. This downsampling method sampled rows from training data in such a way that the majority level (i.e., when the products were not backordered) is sampled less. But let's check how the results would change if we didn't downsample. The results when downsampling is not used are represented in Table 12.5, along with the confusion matrix in Table 12.6.

.metric	**.estimator**	**.estimate**	**.config**
accuracy	binary	0.8602794	Preprocessor1_Model1
roc_auc	binary	0.9038896	Preprocessor1_Model1

	No	**Yes**
No	393	61
Yes	9	38

We notice that both accuracy and AUC values are high and comparable to the earlier analysis when we did downsampling. However, when we estimate sensitivity and specificity, we observe a very different picture. Without downsampling, sensitivity is $\frac{38}{38+61} = 0.384$, and specificity is $\frac{393}{393+9} = 0.978$. There is a big drop in sensitivity from 0.838 to 0.384 and an increase in specificity from 0.818 to 0.978. This shows that in the absence of downsampling, our model is not able to identify products that will eventually be backordered. Downsampling addresses this weakness, and the

cost we pay is a drop in specificity. Downsampling ensures that without compromising on AUC and accuracy values, we can make sensitivity and specificity very comparable. Note that downsampling is one of the many subsampling methods that are available in analysis packages (see www.tidymodels.org/learn/models/sub-sampling/ for other subsampling methods that can be used).

12.7 BUSINESS INSIGHTS AND SUMMARY

In this chapter, we examined backorders, a major supply chain topic that can cause multiple issues such as dissatisfaction, reduced purchase probability, and reduced profitability. In order to predict potential backorder issues, we used random forests to build a predictive model. Random forests belong to the larger set of decision tree models that are sometimes preferred over opaque predictive models because they allow some transparency about model working. They also are preferred since they have the ability to work well with different types of categorical and continuous predictor variables, multicollinear data, and outliers.

The random forest model includes 16 predictor variables to predict whether or not a product is likely to be backordered. Since we relied on a categorical variable (*yes* vs. *no*) as the outcome variable, we use a confusion matrix to examine the accuracy, specificity, and sensitivity of the random forest model. We also learned that products that are likely to be backordered form a smaller proportion of the dataset than those that will not be backordered. We then understood how the downsampling method can be used to address the issue of imbalanced data. In addition, we discussed how we can check for whether the accuracy of the model is changed because of downsampling.

12.8 IMPLEMENTATION USING R: BACKORDER PREDICTION

The outcome variable was whether the product went on backorder or not. The random forest was provided with 16 predictor variables such as the following:

went_on_backorder = the outcome variable of whether a product was backordered or not and could take the value of *yes* or *no*

national_inv = indicates the current level of inventory

lead_time = measures the transit time for the product

in_transit_qty = indicates how much of the product is currently in transit

forecast_3_month, forecast_6_month, forecast_9_month = three different predictors capturing the sales forecast for the coming 3, 6, and 9 months

sales_1_month, sales_3_month, sales_6_month, sales_9_month = four variables indicating sales of the product in the last 1, 3, 6, and 9 months

min_bank = the minimum amount of product that is recommended to keep in stock

pieces_past_due = captures whether any parts are overdue from the supplier

perf_6_month_avg, perf_12_month_avg = two variables representing the product performance in the past 6 and 12 months

local_bo_qty = captures the amount of orders overdue

deck_risk, ppap_risk = two different types of risk in delivery represented in a binary form of *yes* and *no*

We first read in the data and the libraries such as `readxl`,[6] `tidymodels`,[7] `dplyr`,[8] `ranger`,[9] and `themis`.[10]

After loading the required packages, we import the data and check the class of nonnumeric variables. We want to ensure that nonnumeric variables such as `deck_risk`, `ppap_risk`, and `went_on_backorder` are categorical variables and will be recognized as `factor` by R.

```
set.seed(12345)
library(readxl)
library(tidymodels)
library(dplyr)
library(ranger)
library(themis)

library(ggplot2)

library(doParallel)


back_order_data <-read.csv(url
("http://data.mishra.us/files/chapter_decision_uncertainty/backorder.csv"))

sapply(back_order_data,class)
##      national_inv                lead_time             in_transit_qty       forecast_3_month
##         "integer"                 "integer"                  "integer"              "integer"
##   forecast_6_month       forecast_9_month           sales_1_month          sales_3_month
##         "integer"                 "integer"                  "integer"              "integer"
##     sales_6_month          sales_9_month              min_bank           pieces_past_due
##         "integer"                 "integer"                  "integer"              "integer"
##   perf_6_month_avg      perf_12_month_avg            local_bo_qty                deck_risk
##         "numeric"                 "numeric"                  "integer"              "character"
##         ppap_risk      went_on_backorder
##         "character"               "character"
```

Since these categorical variables are appearing as characters, we convert them into factors and check again their class assignment.

```
back_order_data <- as.data.frame(unclass(back_order_data),
                                 stringsAsFactors = TRUE)
sapply(back_order_data,class)
```

```
##  national_inv      lead_time         in_transit_qty    forecast_3_month
##       "integer"         "integer"         "integer"         "integer"
##  forecast_6_month  forecast_9_month  sales_1_month     sales_3_month
##       "integer"         "integer"         "integer"         "integer"
##  sales_6_month     sales_9_month     min_bank          pieces_past_due
##       "integer"         "integer"         "integer"         "integer"
##  perf_6_month_avg  perf_12_month_avg local_bo_qty      deck_risk
##       "numeric"         "numeric"         "integer"         "factor"
##  ppap_risk went_on_backorder
##       "factor"          "factor"
```

As we discussed earlier, backorder data tend to have a class imbalance, where the number of items that were backordered is proportionately less than the number of items that were not backordered. Let's check this in our data through a simple plot, shown in Figure 12.6.

```
ggplot(back_order_data, aes(went_on_backorder))+
 xlab("Backorder")+ylab("Number of data points")+
 geom_bar()+
 theme_minimal()
```

The plot shows that items that were backordered (i.e., when the outcome variable `went_on_backorder = yes`) represented approximately $\frac{1}{5}$ of the data. This makes `went_on_backorder = no` the majority level (also referred to as majority class) and `went_on_backorder = yes` a minority level. It is necessary to use a sampling method that provides appropriate representation to minority level.

As the first step to build a model to predict backorder, we split the data into train and test groups. Let's use 75% data for training and 25% for testing the model.

```
backorder_split <- initial_split(back_order_data, strata =
went_on_backorder,
         prop = 0.75)
backorder_train <- training(backorder_split)
backorder_test <- testing(backorder_split)
```

Next we define the model formulation in `recipe` and indicate that `went_on_backorder` is the outcome variable and all other variables are predictors.

To handle the issue of class imbalance, we use `step_downsample()`. This function samples rows from training data in such a way that the majority level (i.e., `went_on_backorder = no`) is sampled less. This increases the proportion of the minority level (i.e., `went_on_backorder = yes`) in model training. However, it is important to note that such undersampling does not occur in the test data, which ensures that model performance is tested in data that closely represent the real world (i.e., where one class is overrepresented). `under_ratio = 1.5` implies that majority levels will have at most 1.5 times more rows than minority levels. If we compare this situation to our base situation when no sampling is applied, we will have majority level with 4 times as many rows as minority levels.

```
backorder_recipe <- recipe(went_on_backorder ~ ., data =
backorder_train)%>%
  step_downsample(went_on_backorder,under_ratio = 1.5)
```

Using `rand_forest()`, we specify the random forest algorithm and also the hyperparameters such as `mtry` (the number of predictors randomly sampled at each split when a tree is created), `trees` (the number of trees contained in a random forest), and `min_n` (the minimum number of data points in a node that is required for the node to be split further). `tune()` specifies that each hyperparameter needs to be tuned though grid search. `set_engine("ranger")` specifies that our computational engine is `ranger`, a fast implementation of random forests. Since our outcome variable is categorical (*yes* vs. *no*), `set_mode("classification")` specifies that our model type is a classification model.

```
model_backorder <- rand_forest(
  mtry = tune(),
  trees = tune(),
  min_n = tune()) %>%
  set_mode("classification") %>%
   set_engine("ranger")

backorder_folds <- vfold_cv(backorder_train, v = 5)
```

Next, we aggregate the recipe and model using `workflow(). add_model` adds the `model_backorder` specified in the last step with `backorder_recipe`, which contains information about predictor and outcome variables and dummy coding.

```
backorder_wf <- workflow() %>%
  add_model(model_backorder) %>%
  add_recipe(backorder_recipe)
```

We next specify the cross-validation process using `vfold_cv(). V = 5` specifies that there will be five partitions of the training dataset and that each hyperparameter combination's performance will be assessed using this cross-validation procedure. `doParallel:: registerDoParallel(cores = 10)` registers that 10 cores will be used for parallel execution of the model. We can change the values of the cores depending on the computational resources available.

`grid = 50` in `tune_grid()` specifies that 50 different hyperparameter combinations should be created. `resamples = backorder_folds` uses cross-validation specified in `vfold_cv()` and helps to avoid overfitting the training data. For each of the 50 combinations of hyperparameters, a model will be fit on the four partitions, and its performance will be assessed on the fifth partition.

```
backorder_folds <- vfold_cv(backorder_train, v = 5)
doParallel::registerDoParallel(cores = 10)
backorder_tune <-
  backorder_wf %>%
  tune_grid(
```

```
    resamples = backorder_folds,
    grid = 50
  )
```

We use `select_best("roc_auc")` to pick the best hyperparameter combination.

```
best_model <- backorder_tune %>%
       select_best("roc_auc")
```

```
best_model
```

```
## # A tibble: 1 × 4
##        mtry trees min_n .config
##       <int> <int> <int> <chr>
## 1        5    69    31 Preprocessor1_Model20
```

The analysis reveals that the best hyperparameter combination is `mtry` = 5, `tree` = 69, and `min_n` = 31. Now we will use these hyperparameter values to update the workflow using `finalize_workflow`. `finalize_workflow` adds the best model to the workflow `backorder_wf` created earlier. This gives us the `final_workflow` that captures all the information about the model, hyperparameters, outcome variable, and predictors.

```
final_workflow <-
 backorder_wf %>%
 finalize_workflow(best_model)
```

We fit the model using the best hyperparameter combination on the entire training set and then validate its performance on the test data. `last_fit` helps us accomplish this. `split = backorder_split` specifies that `backorder_split` contains information about the test and train split performed earlier. Using `collect_metrics`, we report the accuracy and ROC AUC values. Finally using `conf_mat()`, we produce a confusion matrix.

```
final_fit <-
  final_workflow %>%
  last_fit(split = backorder_split)
```

```
final_fit %>%
  collect_metrics()
```

```
## # A tibble: 2 × 4
##   .metric  .estimator .estimate .config
##   <chr>    <chr>          <dbl> <chr>
## 1 accuracy binary         0.822 Preprocessor1_Model1
## 2 roc_auc  binary         0.908 Preprocessor1_Model1
```

```
final_fit %>%
```

```
collect_predictions()%>%
```

```
conf_mat(truth = went_on_backorder, estimate = .pred_class)
```

```
##                Truth
## Prediction        No  Yes
## No               329   16
## Yes               73   83
```

The results show that the accuracy is 82.2% and AUC is 0.908. Given the imbalance in levels of the outcome variable and the fact that the minority level (i.e., `went_on_backorder = yes`) is more important for the prediction task, we estimate sensitivity as well as specificity of this prediction. We use Table 12.4 and the formula for sensitivity and specificity defined earlier for our calculations.

For our model, sensitivity is $\frac{83}{83+16} = 0.838$, and specificity is $\frac{329}{329+73} = 0.818$.

12.9 UNDERSTANDING THE CHAPTER

1. Given a dataset where instances of backorders are vastly outnumbered by regular orders, hyperparameter tuning in Random Forest, such as the number of trees or minimum data points per node, can be used. How might adjusting these parameters specifically aid or not aid in improving prediction accuracy for the minority class (backorders)?
2. Consider a predictive modeling task involving a Random Forest classifier. Explain the role and significance of employing *k*-fold cross-validation (CV) in the model evaluation process. How does *k*-fold CV work, and in what ways does it attempt to provide a robust estimate of the model's performance? Moreover, discuss the trade-offs involved in choosing a specific *k* value (e.g., *k* = 5 vs. *k* = 10).
3. Decisions that are made with some information missing are called which of the following?
    a. Risky decisions
    b. Decisions under certainty
    c. No decision can be made
    d. Binary decisions
4. From the options provided, which are features of decisions made under uncertainty? (Choose all that apply.)
    a. Weighing the risks of making the decision versus the benefits of the decision
    b. Weighing the risks versus the costs of making the decision
    c. Weighing the benefits of the decision versus profits from the sales
    d. Evaluating existing options versus new ones
5. Which of the following statements is *true* about the bullwhip effect?
    a. It occurs when a small change in demand is overestimated, resulting in oversupply.
    b. It occurs when a small change in supply is underestimated.
    c. It occurs when demand matches supply.
    d. It occurs when inflation causes demand to fall.
6. For decision trees, the metric used to find which variable is a better classifier is which of the following?
    a. Gini impurity

b. Coefficient of determination
c. Variance inflation factor (VIF)
d. Cophenetic correlation coefficient

7. Which of the following statements about bagging is *false*?
    a. Bagging is bootstrap aggregation.
    b. Bagging involves sampling with replacement.
    c. Bagging involves sampling without replacement.
    d. Bagging helps build diverse trees.
8. Which of the following statements about downsampling is *false*?
    a. It addresses the issue of data imbalance.
    b. It allows the model to better learn the features of the minority class.
    c. It severely reduces accuracy of the model.
    d. It slightly decreases specificity but improves sensitivity significantly.
9. Let's assume that we are using a decision tree to find out which predictor variable(s) can better classify the outcome variable: whether or not customers will buy a product. Given the following Gini impurity values for the three predictor variables—Predictor A = 0.55, Predictor B = 0.24, and Predictor C = 0.76—which of the following options would be considered the optimal classifier of the buy/no buy decision?
    a. Predictor A
    b. Predictor B
    c. Predictor C
    d. Both Predictor A and Predictor C

ENDNOTES

1. Heath, Chip, and Amos Tversky, "Preference and Belief: Ambiguity and Competence in Choice Under Uncertainty," *Journal of Risk and Uncertainty* 4, no. 1 (1991): 5–28.

2. Fischhoff, Baruch, and Alex L. Davis, "Communicating Scientific Uncertainty," *Proceedings of the National Academy of Sciences* 111, Suppl. 4 (2014): 13664–13671.

3. IBM Corporation, "Scientific and Technical Report: Backorder Prediction," for Defense Logistics Agency Research and Development (Contract No. SP4701-20-C-0052), submitted February 26, 2021, https://apps.dtic.mil/sti/pdfs/AD1141471.pdf.

4. Bhan, Hyung Sup, and Eric T. Anderson, "Multiyear Impact of Backorder Delays: A Quasi-experimental Approach," *Marketing Science* 22, no. 2 (2022): 314–335.

5. The data for the analysis is a subset of data adopted from Santis et al. (2017). It was made available as a Kaggle competition and is posted at https://github.com/rodrigosantis1/backorder_prediction. Santis, Rodrigo B., Eduardo Pestana de Aguiar, and Leonardo Goliatt, "Predicting Material Backorders in Inventory Management Using Machine Learning," Fourth IEEE Latin American Conference on Computational Intelligence, Arequipa, Peru (2017).

6. Wickham, Hadley, and Jennifer Bryan, "readxl: Read Excel Files," R package version 1.4.3 (July 6, 2023), https://CRAN.R-project.org/package=readxl.

7. Kuhn, Max, et al., "Tidymodels: A Collection of Packages for Modeling and Machine Learning Using tidyverse Principles" (2020), https://www.tidymodels.org.

8. Wickham, Hadley, Romain François, Lionel Henry, and Kirill Müller, "dplyr: A Grammar of Data Manipulation," R package version 1.1.3 (September 3, 2023), https://CRAN.R-project.org/package=dplyr.

9. Wright, Marvin N., and Andreas Ziegler, “ranger: A Fast Implementation of Random Forests for High Dimensional Data in C++ and R,” *Journal of Statistical Software* 77, no. 1 (2017): 1–17, https://doi.org/10.18637/jss.v077.i01.

10. Hvitfeldt, Emil, “themis: Extra Recipes Steps for Dealing With Unbalanced Data,” R package version 1.0.0 (2022), https://CRAN.R-project.org/package=themis.

Descriptions of Images and Figures

Back to Figure

The figure consists of images from left to right as follows: image for human, house, industry and vehicles for construction. Above human is the description CUSTOMER DEMAND INCREASE; above house is the description RETAILER OVERORDERING; above industry is the description OVER MANUFACTURING; above the vehicle is the description SUPPLY. In the figure, from left to right is a dotter wave enclosing the image and the description is present above each peak.

Back to Figure

The figure has a horizontal line on the left and above the line is a vertical line. On the left of the line are four circular nodes of which first three are grey and the last is black in colour and this region is labeled Low. On the right are five nodes of which first four are black and the last is grey in color and this region is labeled High. The left image is labeled Forecasted Sales at the bottom. On the right is an image with a horizontal line and vertical line above the horizontal line labeled Transit Time. On the left is the region Low with five circular nodes of which first two nodes are grey and last three nodes are black. On the right is the region for high with four nodes of which first two nodes are black and last two nodes are grey.

Back to Figure

The figure has two images. On the top is an image for Forecasted Sales, which has a horizontal line and a vertical dotted line on the top. The left side is a region for Low with three grey nodes on the left and a black node on the right, and on the right is the region for high with four black nodes on the left and a grey node on the right. At the bottom is the equation *Weighted Average Impurity* = (5/9) 0.32 + (4/9) 0.375 = 0.34. On the top left is the $1-(1/4)^2-(3/4)^2$ = 0.375 and on the right is the equation 1–(4/5)2−(1/5)2 = 0.32. At the bottom is an image for Transit Time, which has a horizontal line and a vertical dotted line on the top. The left side is a region for Low with two grey nodes on the left and three black node on the right, and on the right is the region for high with two blacks nodes on the left and two grey node on the right. At the bottom is the equation *Weighted Average Impurity* = (5/9) 0.48 + (4/9) 0.5 = 0.48. On the top left is the equation $1-(2/5)^2-(3/5)^2$ = 0.48 and on the right is the equation 1–(2/4)2−(2/4)2 = 0.5.

Back to Figure

The figure is a flow diagram consisting of rectangular blocks and circular nodes in the box. On top is a block with four black nodes on the top and five grey nodes at the bottom. On the left is a block labeled Low with a black node on the top and three grey nodes at the bottom. This block is again divided into two blocks labeled Low on the left and High on the right, and the term Transit Time is at the center. On the left is a block with a black node on the top and two grey nodes on the right. On the right is a block for high with four black nodes on the top and a grey node at the bottom. This block is divided as Low with two black nodes on the left and High on the right with two black nodes on the top and a grey node at the bottom and at the center is the term Transit Time.